Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Scripting Language

BCA — Semester 4 — 2020 — Answer Sheet

Subject Code: CACS254

Group B

Attempt any SIX questions. [6×5=30]

1. What is javaScript? What are the potential platforms where javaScript can be used?

[1+3]

Answer: JavaScript: JavaScript is a lightweight, interpreted, object-oriented programming language with first-class functions. It was created by Brendan Eich at Netscape in 1995 and is best known as the scripting language for Web pages. Despite the name similarity, it is not related to Java. It conforms to the ECMAScript specification. **Potential Platforms:** 1. **Web Browsers (Client-Side):** DOM manipulation, form validation, animations, AJAX requests, SPAs (React, Vue, Angular) 2. **Server-Side (Node.js):** Building scalable network applications, REST APIs, real-time chat, microservices 3. **Mobile Applications:** React Native, Ionic, NativeScript for cross-platform mobile apps 4. **Desktop Applications:** Electron (VS Code, Slack, Discord), NW.js 5. **IoT and Embedded:** Johnny-Five, Cylon.js for robotics and hardware 6. **Game Development:** Browser-based games using Canvas, WebGL, Phaser.js, Three.js 7. **Machine Learning:** TensorFlow.js for running ML models in browser 8. **Cloud and Serverless:** AWS Lambda, Azure Functions with Node.js runtime 9. **Database:** MongoDB uses JavaScript for queries and MapReduce operations

2. Define javaScript event. Illustrate how an event handler response particular event in web page.

[1+4]

Answer: JavaScript Event: An event is an action or occurrence that happens in the browser, which the browser can respond to. Events are the foundation of interactive web applications. They can be triggered by user actions (click, keypress, mouseover), browser actions (page load, resize, scroll), or API-generated notifications. **Common Events:** click, dblclick, mouseover, mouseout, keydown, keyup, submit, load, resize, scroll, focus, blur **Event Handler Example:** <!-- HTML --> <button id="myBtn">Click Me</button> <script> // Method 1: Inline (not recommended) // <button onclick="alert('Hello')"> // Method 2: DOM property document.getElementById('myBtn').onclick = function() { document.getElementById('output').textContent = 'Button was clicked!'; }; // Method 3: addEventListener (recommended) document.getElementById('myBtn').addEventListener('click', function(event) { console.log('Event type:', event.type); console.log('Target element:', event.target); document.getElementById('output').textContent = 'Clicked at coordinates: ' + event.clientX + ', ' + event.clientY; }); </script> **Event Flow:** Events propagate through the DOM in two phases: • **Event Capturing:** Event travels from document root down to the target • **Event Bubbling:** Event bubbles up from target to document root **Event Object:** When an event occurs, the browser creates an event object containing information about the event (type, target, timestamp, mouse coordinates, key codes, etc.). This object is passed to the event handler function.

3. What is an Immediately Invoked Function in javaScript? Explain with suitable example.

[1+4]

Answer: Immediately Invoked Function Expression (IIFE): An IIFE is a JavaScript function that runs as soon as it is defined. It is a design pattern that produces a lexical scope using JavaScript's function scoping. IIFEs are commonly used to avoid polluting the global namespace, create private variables, and implement the module pattern. **Syntax:** (function() { // code here runs immediately })(); // Or with arrow function (() => { // code here runs immediately })(); **Example 1: Basic IIFE** (function() { var message = "Hello from IIFE!"; console.log(message); // Outputs: Hello from IIFE! })(); console.log(message); // Error: message is not defined **Example 2: IIFE with Parameters** (function(name, age) { console.log(name + " is " + age + " years old."); })("Ram", 20); // Outputs: Ram is 20 years old. **Example 3: Module Pattern (Practical Use)** var counterModule = (function() { var count = 0; // Private variable return { increment: function() { count++; return count; }, decrement: function() { count--; return count; }, getCount: function() { return count; } }; })(); console.log(counterModule.getCount()); // 0 counterModule.increment(); console.log(counterModule.getCount()); // 1 console.log(counterModule.count); // undefined (private!) **Benefits:** Encapsulation, private variables, avoiding global scope pollution, isolation of variables in loops.

4. Explain an array using suitable example and discuss how foreach loop accessing of elements is stored in an array using PHP.

[2+3]

Answer: Array: An array is a data structure that stores multiple values in a single variable. In JavaScript, arrays are dynamic, zero-indexed, and can hold mixed data types. They are implemented as objects with integer-based keys and have built-in methods for manipulation. **JavaScript Array Example:** let fruits = ["Apple", "Banana", "Orange"]; console.log(fruits[0]); // "Apple" console.log(fruits.length); // 3 fruits.push("Mango"); // Add element fruits.pop(); // Remove last element **PHP foreach Loop:** In PHP, foreach is a convenient way to iterate over arrays without managing counters. **Indexed Array with foreach:** <?php $students = ["Ram", "Sita", "Hari"]; // foreach with values only foreach ($students as $name) { echo $name . "
"; } // Output: Ram Sita Hari // foreach with index and value foreach ($students as $index => $name) { echo "Index $index: $name
"; } // Output: Index 0: Ram, Index 1: Sita, Index 2: Hari ?> **Associative Array with foreach:** <?php $student = ["name" => "Ram", "age" => 20, "faculty" => "BCA"]; foreach ($student as $key => $value) { echo "$key: $value
"; } // Output: name: Ram, age: 20, faculty: BCA ?> **How foreach accesses elements:** PHP foreach creates a copy of the array's internal pointer and iterates through each element. For indexed arrays, it accesses elements by position (0, 1, 2...). For associative arrays, it accesses key-value pairs using the hash table implementation. It does not require manual index management like traditional for loops.

5. What is jQuery? Illustrate different types of selectors in jQuery with appropriate example?

[1+4]

Answer: jQuery: jQuery is a fast, small, and feature-rich JavaScript library created by John Resig in 2006. It simplifies HTML document traversal and manipulation, event handling, animation, and Ajax interactions. Its motto is "Write less, do more." jQuery abstracts browser inconsistencies and provides a consistent API across all major browsers. **jQuery Selectors:** jQuery uses CSS-style selectors to find and select HTML elements. **1. Element Selector:** $('p') // Selects all elements $('div') // Selects all elements **2. ID Selector:** $('#header') // Selects element with id="header" $('#menu').hide(); // Hides the menu **3. Class Selector:** $('.active') // Selects all elements with class="active" $('.btn').click(function() { ... }); **4. Universal Selector:** $('*') // Selects all elements **5. Multiple Selectors:** $('p, div, .highlight') // Selects all p, div, and .highlight elements **6. Descendant Selector:** $('div p') // Selects all inside $('#nav a') // Selects all links inside #nav **7. Child Selector:** $('ul > li') // Selects direct <li> children of <ul> **8. Attribute Selectors:** $('[href]') // Elements with href attribute $('input[type="text"]') // Text inputs $('[class*="col"]') // Classes containing "col" **9. Pseudo-class Selectors:** $('li:first') // First <li> $('li:last') // Last <li> $('tr:even') // Even rows $('tr:odd') // Odd rows $(':input') // All form inputs **Complete Example:** Welcome Details here <button class="btn">Click</button> <script> $(document).ready(function() { $('.intro').css('color', 'blue'); $('#container').addClass('highlight'); $('button').click(function() { $('p').toggle(); }); }); </script>

6. What is the benefit of using AJAX? Discuss major methods that communicate with server-side scripting.

[1+4]

Answer: AJAX (Asynchronous JavaScript and XML): AJAX is a technique for creating fast, dynamic web pages by allowing web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes. This means parts of a web page can be updated without reloading the entire page. **Benefits of AJAX:** 1. **Improved User Experience:** No full page reloads — updates happen seamlessly 2. **Faster Response:** Only necessary data is transferred, reducing bandwidth 3. **Reduced Server Load:** Less data processing and rendering on server side 4. **Asynchronous Operations:** User can continue interacting while data loads 5. **Rich Interactive Applications:** Enables features like auto-complete, infinite scroll, live search 6. **Partial Page Updates:** Only affected parts of DOM are updated **Major Methods for Server Communication:** 1. **XMLHttpRequest (Legacy):** var xhr = new XMLHttpRequest(); xhr.open('GET', 'api/data.php', true); xhr.onreadystatechange = function() { if (xhr.readyState === 4 && xhr.status === 200) { console.log(xhr.responseText); } }; xhr.send(); 2. **Fetch API (Modern):** fetch('api/data.php') .then(response => response.json()) .then(data => console.log(data)) .catch(error => console.error(error)); 3. **jQuery AJAX:** $.ajax({ url: 'api/data.php', method: 'POST', data: { name: 'Ram', age: 20 }, success: function(response) { console.log(response); }, error: function(xhr, status, error) { console.log(error); } }); // Shorthand methods $.get('api/data.php', function(data) { ... }); $.post('api/save.php', { name: 'Ram' }, function(data) { ... }); $.getJSON('api/data.json', function(data) { ... }); 4. **Axios (Popular Library):** axios.get('/api/users') .then(response => console.log(response.data)) .catch(error => console.error(error)); axios.post('/api/users', { name: 'Ram' }) .then(response => console.log(response.data)); **HTTP Methods Used:** • **GET:** Retrieve data from server • **POST:** Submit data to server • **PUT:** Update existing resource • **DELETE:** Remove resource • **PATCH:** Partial update

7. Discuss about MVC architecture with appropriate diagram.

[5]

Answer: MVC (Model-View-Controller) Architecture: MVC is a software design pattern that separates an application into three interconnected components, enabling modular development, easier maintenance, and parallel work. **Components:** 1. **Model:** • Represents the data and business logic • Manages data storage, retrieval, and validation • Notifies views when data changes • Independent of user interface • Example: User class with properties (name, email) and methods (save(), validate()) 2. **View:** • Represents the user interface and presentation layer • Displays data from the model to the user • Sends user commands to the controller • Should not contain business logic • Example: HTML templates, React components, blade files 3. **Controller:** • Acts as an intermediary between Model and View • Receives user input from View • Processes input, interacts with Model • Selects appropriate View for response • Example: UserController with methods like index(), store(), show() **Workflow:** 1. User interacts with View (clicks button, submits form) 2. View sends request to Controller 3. Controller processes request, updates/retrieves Model 4. Model performs business logic and data operations 5. Model returns data to Controller 6. Controller selects View and passes data 7. View renders data for user **Advantages:** • Separation of concerns — each component has distinct responsibility • Parallel development — designers work on Views, developers on Models • Easier testing — components can be unit tested independently • Reusability — same model can serve multiple views • Maintainability — changes to UI don't affect business logic **Examples:** Laravel (PHP), Ruby on Rails, Django (MTV variant), ASP.NET MVC, Spring MVC.

■ *Diagram Required:* MVC architecture diagram: User interacts with View (UI). View sends input to Controller. Controller updates Model. Model notifies View of changes. View displays updated data to User. Three boxes labeled Model, View, Controller with arrows showing this flow.

Group C

Attempt any TWO questions. [2×10=20]

8. a) Write a program which includes a function sum(). This function sum() should be designed to add an arbitrary list of parameters. b) Write a program that displays the continuous time in the web page. The time should be in the format of HH:MM:SS.

[5+5]

Answer: a) JavaScript Function with Arbitrary Parameters: // Using rest parameters (...args) function sum(...numbers) { let total = 0; for (let num of numbers) { total += num; } return total; } // Alternative using arguments object function sumClassic() { let total = 0; for (let i = 0; i < arguments.length; i++) { total += arguments[i]; } return total; } // Alternative using reduce function sumReduce(...numbers) { return numbers.reduce((acc, curr) => acc + curr, 0); } // Test cases console.log(sum(1, 2)); // Output: 3 console.log(sum(1, 3, 4)); // Output: 8 console.log(sum(10, 20, 30, 40)); // Output: 100 console.log(sum()); // Output: 0 **HTML Integration:** <!DOCTYPE html> <html> <body> <script> function sum(...nums) { return nums.reduce((a, b) => a + b, 0); } document.getElementById('result').textContent = 'sum(1,2) = ' + sum(1,2) + ', sum(1,3,4) = ' + sum(1,3,4); </script> </body> </html> **b) Continuous Time Display Program:** <!DOCTYPE html> <html> <head> <style> #clock { font-size: 48px; font-family: 'Courier New', monospace; color: #0ea5e9; text-align: center; margin-top: 100px; } </style> </head> <body> 00:00:00 <script> function updateClock() { const now = new Date(); let hours = now.getHours(); let minutes = now.getMinutes(); let seconds = now.getSeconds(); // Add leading zeros hours = hours < 10 ? '0' + hours : hours; minutes = minutes < 10 ? '0' + minutes : minutes; seconds = seconds < 10 ? '0' + seconds : seconds; const timeString = hours + ':' + minutes + ':' + seconds; document.getElementById('clock').textContent = timeString; } // Update immediately and then every second updateClock(); setInterval(updateClock, 1000); </script> </body> </html> **Explanation:** The setInterval function calls updateClock every 1000 milliseconds (1 second). The Date object provides current time, which is formatted with leading zeros using the ternary operator. The display updates continuously without page refresh.

9. Define method overriding. Write a PHP program to handle the overriding situation.

[2+8]

Answer: Method Overriding: Method overriding is an object-oriented programming feature where a subclass provides a specific implementation of a method that is already defined in its parent class. The overridden method in the child class has the same name, same parameters, and same return type as the method in the parent class. It enables runtime polymorphism and allows subclasses to customize or extend the behavior inherited from parent classes. **Rules for Method Overriding:** • Method name must be identical to parent method • Parameter list must be the same • Access level cannot be more restrictive • Use "parent::methodName()" to call parent's method **PHP Program Demonstrating Method Overriding:** <?php class Animal { protected $name; public function __construct($name) { $this->name = $name; } // Parent method to be overridden public function makeSound() { return "Some generic animal sound"; } public function describe() { return "This is a " . $this->name; } } class Dog extends Animal { public function __construct($name) { parent::__construct($name); } // Overriding makeSound method public function makeSound() { return "Woof! Woof!"; } // Overriding with additional functionality public function describe() { // Call parent's method and extend it return parent::describe() . " and it barks loudly."; } } class Cat extends Animal { public function __construct($name) { parent::__construct($name); } // Overriding makeSound method public function makeSound() { return "Meow! Meow!"; } } // Usage $dog = new Dog("German Shepherd"); echo $dog->describe() . "
"; echo "Sound: " . $dog->makeSound() . "

"; $cat = new Cat("Persian Cat"); echo $cat->describe() . "
"; echo "Sound: " . $cat->makeSound() . "
"; ?> **Output:** This is a German Shepherd and it barks loudly. Sound: Woof! Woof! This is a Persian Cat Sound: Meow! Meow! **Real-World Example — Payment Gateway:** <?php abstract class PaymentGateway { abstract public function processPayment($amount); public function validate($amount) { return $amount > 0; } } class EsewaPayment extends PaymentGateway { public function processPayment($amount) { return "Processing Rs. $amount via eSewa... Success!"; } } class KhaltiPayment extends PaymentGateway { public function processPayment($amount) { return "Processing Rs. $amount via Khalti... Success!"; } } ?>

10. Design following form in HTML and write corresponding PHP and SQL code for CRUD operations based on user's data. Assume your own database and database connectivity related constraints if necessary.

[10]

Answer: HTML Form Design: <!DOCTYPE html> <html> <head><title>User Management</title></head> <body> <h2>User Registration Form</h2> <form action="process.php" method="POST"> <input type="hidden" name="action" value="create"> Name: <input type="text" name="name" required>

 Email: <input type="email" name="email" required>

 Phone: <input type="text" name="phone">

 Address: <textarea name="address"></textarea>

 <input type="submit" value="Submit"> </form> </body> </html> **SQL Database Setup:** CREATE DATABASE userdb; USE userdb; CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, phone VARCHAR(20), address TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP); **Database Connection (db.php):** <?php $host = "localhost"; $user = "root"; $pass = ""; $dbname = "userdb"; $conn = new mysqli($host, $user, $pass, $dbname); if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); } ?> **PHP CRUD Operations (process.php):** <?php include 'db.php'; $action = $_POST['action'] ?? $_GET['action'] ?? ''; // CREATE if ($action == 'create') { $name = $conn->real_escape_string($_POST['name']); $email = $conn->real_escape_string($_POST['email']); $phone = $conn->real_escape_string($_POST['phone']); $address = $conn->real_escape_string($_POST['address']); $sql = "INSERT INTO users (name, email, phone, address) VALUES ('$name', '$email', '$phone', '$address')"; if ($conn->query($sql) === TRUE) { echo "User registered successfully!
"; } else { echo "Error: " . $conn->error; } } // READ elseif ($action == 'read') { $sql = "SELECT * FROM users"; $result = $conn->query($sql); echo "<table border='1'>"; echo "<tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th></tr>"; while ($row = $result->fetch_assoc()) { echo "<tr>"; echo "<td>" . $row['id'] . "</td>"; echo "<td>" . $row['name'] . "</td>"; echo "<td>" . $row['email'] . "</td>"; echo "<td>" . $row['phone'] . "</td>"; echo "</tr>"; } echo "</table>"; } // UPDATE elseif ($action == 'update') { $id = intval($_POST['id']); $name = $conn->real_escape_string($_POST['name']); $email = $conn->real_escape_string($_POST['email']); $sql = "UPDATE users SET name='$name', email='$email' WHERE id=$id"; if ($conn->query($sql) === TRUE) { echo "User updated successfully!"; } else { echo "Error: " . $conn->error; } } // DELETE elseif ($action == 'delete') { $id = intval($_GET['id']); $sql = "DELETE FROM users WHERE id=$id"; if ($conn->query($sql) === TRUE) { echo "User deleted successfully!"; } else { echo "Error: " . $conn->error; } } $conn->close(); ?> **Note:** For production, use prepared statements to prevent SQL injection: $stmt = $conn->prepare("INSERT INTO users (name, email) VALUES (?, ?)"); $stmt->bind_param("ss", $name, $email); $stmt->execute();

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.